PRACTICAL – 06

Aim :

To implement a **remote control system in Java** using an interface with on(), off(), and a default toggle() method, and to demonstrate **functional interfaces, anonymous classes, and lambda expressions** for controlling device switching based on the hour.

CODE :

```
interface Switchable {
     void on();
    void off();
    default void toggle() {
        on();
    }
}
class Fan implements Switchable {

    public void on() {
        System.out.println("Fan is ON");
    }

    public void off() {
        System.out.println("Fan is OFF");
    }
}
class Light implements Switchable {
     public void on() {
        System.out.println("Light is ON");
    }
    public void off() {
        System.out.println("Light is OFF");
    }
}
@FunctionalInterface
interface Permission {

    boolean maySwitchOn(Switchable device, int hour);
}
public class Main {
    public static void main(String[] args) {

        Switchable fan = new Fan();
        Switchable light = new Light();

        Switchable[] devices = {fan, light};

        for (int i = 0; i < devices.length; i++) {
```

```
            devices[i].toggle();
        }

        Permission p1 = new Permission() {

            public boolean maySwitchOn(Switchable device, int hour) {
                return hour >= 6 && hour <= 22;
            }
        };

        System.out.println("Anonymous class:");
        System.out.println(p1.maySwitchOn(fan, 10));
        System.out.println(p1.maySwitchOn(fan, 23));

        Permission p2 =
                (device, hour) -> hour >= 6 && hour <= 22;
        System.out.println("Lambda:");
        System.out.println(p2.maySwitchOn(light, 15));
        System.out.println(p2.maySwitchOn(light, 2));
    }
}
```

OUTPUT :

```
Fan is ON
Light is ON
Anonymous class:
true
false
Lambda:
true
false
```

Learning Outcomes :

1. Understand and implement **Java interfaces**.
2. Use **default methods** in an interface.
3. Implement an interface using multiple classes such as Fan and Light.
4. Store different objects in a **Switchable[] array**.
5. Use a **loop** to perform operations on multiple objects.
6. Understand and use **functional interfaces**.
7. Implement a functional interface using an **anonymous class**.
8. Implement the same functional interface using a **lambda expression**.
9. Understand basic **polymorphism** in Java.
10. Apply a condition based on a **device and hour** to decide whether a device can be switched ON.

Aim :

To implement a **notification system in Java** using a functional interface Notifier, lambda expressions for Email and SMS senders, an array to store the senders, and a marker interface Urgent to send urgent notifications twice.

CODE :

```
interface Notifier {

    void send(String message);
}

interface Urgent {
}

public class Main {

    public static void main(String[] args) {

        Notifier email =
                (Notifier & Urgent) (message ->
                        System.out.println("Email: " + message));


        Notifier sms =
                message ->
                        System.out.println("SMS: " + message);

        Notifier[] senders = {email, sms};


        String message = "Exam is tomorrow!";

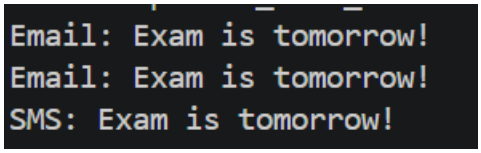
        for (int i = 0; i < senders.length; i++) {

            senders[i].send(message);


            // Send twice if sender is urgent
            if (senders[i] instanceof Urgent) {

                senders[i].send(message);
            }
        }
    }
}
```

OUTPUT :



Learning Outcomes :

1. Understand and implement a **functional interface** in Java.
2. Use the send(String) method to send notifications.
3. Create **Email and SMS senders using lambda expressions**.
4. Store multiple notification senders in an **array**.
5. Use a **loop to broadcast** the same message to all senders.
6. Understand the concept of a **marker interface**.
7. Use the Urgent interface to identify urgent senders.
8. Use instanceof to check whether a sender is marked as urgent.
9. Send a notification **twice for urgent senders**.
10. Apply **lambda expressions and interfaces** in a practical Java application.